

Revisiting Locally Differentially Private Graph Neural Networks

Anonymous Author(s)
Anonymous Institution(s)

Abstract—*Graph neural networks* (GNNs) have been widely adopted in numerous real-world applications. However, training GNNs on raw graph data may raise significant privacy concerns regarding user attributes. *Locally differentially private GNN* (LDPGNN), which perturbs node features before training, represents a promising approach to preserving user privacy while learning from graph data. Although existing LDPGNN methods claim high utility, we observe that they all overlook a simple baseline: training GNNs without using node features and relying solely on graph topology, which we refer to as the *feature-free baseline*. To fill this gap, we reevaluate the representative LDPGNN methods with this baseline across multiple datasets and GNN models. Surprisingly, we find that these methods can hardly outperform the feature-free baseline.

We then delve into the intrinsic reason for this phenomenon by revisiting the LDPGNN’s feature-processing mechanism. Our analysis shows that the server-side “denoising” step improves GNN performance not primarily by recovering the original node features, but by making the resulting node representations largely reflect graph topology. This topology-derived signal becomes the main source of GNN’s utility. Furthermore, we demonstrate that the utility drop under low privacy budgets is not caused by noise destroying useful feature information. Instead, the *unbiased rectification operation*, which is the core component of various LDP mechanisms, introduces a large correction multiplier under low privacy budgets. This multiplier amplifies the magnitude of the inputs fed to the subsequent GNN components, thereby undermining its trainability. As such, the utility drop under low privacy budgets can be easily remedied by removing the unbiased rectification operation. Finally, we apply these insights to reinterpret the state-of-the-art LDPGNN method UPGNET, showing that its utility gains primarily come from improved topology exploitation and the control of input scale, rather than the mechanisms emphasized in the original explanation. Our findings call for a rethinking of how LDPGNN methods should be designed and evaluated.

1. Introduction

Graph neural networks (GNNs) have garnered widespread attention across various fields that require the processing of graph-structured data, such as social science [1], molecular chemistry [2], and bioinformatics [3]. However, training well-performing GNN models oftentimes relies on the collection of sensitive personal data, including user profiles, purchase histories, personal health records. While collecting such data yields better utility

for downstream GNN tasks, it inherently poses severe information leakage risks [4], [5], [6], [7]. To mitigate privacy risks during data collection, *local differential privacy* (LDP) [8] provides a decentralized privacy protecting approach with quantitative and controllable privacy guarantees [9], [10], [11]. Under LDP, users do not send raw sensitive data to the server. Instead, they perturb their data locally before transmission, making the output distributions generated from different inputs statistically similar and thus providing individual-level privacy protection [8]. To this end, recent studies have integrated LDP with GNNs to protect features that containing users’ sensitive attributes during graph learning [12], [13], [14], [15]. This family of methods is referred to as locally differentially private graph neural networks (LDPGNNs).

Although existing LDPGNN methods typically claim high utility, their evaluation overlooks a simple baseline: training GNNs without collecting any sensitive node features and relying only on graph topology, which we refer to as the *feature-free baseline*. This baseline is conceptually necessary because the original purpose of collecting perturbed features is to provide additional information beyond what is already available without feature collection, *i.e.*, the graph topology. We therefore construct a simple feature-free baseline by assigning random features, propagating them over the graph, and feeding the resulting topology-aware representations into downstream GNN tasks. Using this baseline, we reevaluate representative LDPGNN methods across multiple real-world datasets, privacy budgets, and GNN backbones. Surprisingly, we find that existing representative LDPGNN methods *can hardly* outperform the feature-free baseline.

To explain this phenomenon, we revisit the feature-processing procedures of existing LDPGNN methods. We decompose the representation produced by LDPGNNs into three components, corresponding to the graph topology, the raw node features, and the LDP noise, respectively. We theoretically find that the graph topology component *dominates* the information of the representation. To validate this, we further conduct ablation studies that progressively remove the raw node features component and the LDP noise component from the representation. We observe that removing both of these components does not impact the GNN utility, further demonstrating that the GNN utility is mainly sustained by topology-derived information. This explains why LDPGNNs and feature-free baselines achieve similar utility at the *high privacy budget* regime, and it reinterprets the denoising step in LDPGNNs as a procedure

that actually encodes graph topology rather than recovers raw feature information.

We further delve into the intrinsic reason why existing LDPGNNs methods still suffer severe utility drop at the *low privacy budgets* regime. Previous studies explain this utility drop as LDP noise destroying useful feature information. However, if LDPGNNs utility mainly comes from graph topology, it should be independent of the privacy budget. After carefully checking each procedure of LDPGNNs and conducting extensive ablation studies, we surprisingly find the problem arises from the *unbiased rectification operation* widely adopted by LDP mechanisms. Concretely, to obtain an unbiased estimate of raw features, the rectification operation would introduce a multiplier to the perturbed reports. As the privacy budget decreases, this multiplier becomes larger and pushes the representations produced by LDP mechanisms into an excessively large numerical range. This representation amplification disrupts the training of downstream models. As such, we further show that the utility drop under the low privacy budget can be largely remedied by removing the unbiased rectification operation. This finding is counterintuitive, since unbiased rectification is widely used in traditional LDP tasks, such as frequency estimation [8], where the objective is to recover an unbiased estimate of an absolute statistical quantity. In the setting of GNN, however, the downstream tasks are more sensitive to the relative numerical scale of features rather than their absolute numerical scale.

These insights also lead to a different explanation of the state-of-the-art method UPGNET [16], especially its high-order aggregation (HOA) and node feature regularization (NFR) modules. The original explanation argues that HOA improves feature denoising by aggregating higher-order neighborhoods while avoiding over-smoothing, and that NFR reduces feature-estimation error by lowering the effective feature dimension. Our analysis suggests a more direct utility mechanism: HOA improves how graph topology is encoded into node representations, while NFR mitigates the numerical amplification introduced by the unbiased rectification operation. Consequently, the utility gains of UPGNET can be better explained as improved topology exploitation and input-scale control, rather than by the feature-estimation mechanism emphasized in the original explanation.

Contributions. In summary, the main contributions of this work are four-fold:

- We observe the incompleteness of existing LDPGNN methods in evaluation, ignoring a simple baseline that only relying graph topology without using node features. We further reevaluate the representative LDPGNN methods and find that these methods can hardly outperform our proposed feature-free baseline.
- We delve into the intrinsic reason for the performance phenomenon by revisiting the LDPGNN’s feature-processing mechanism. Our theoretical and empirical analysis show that the server-side “denoising” step improves GNN performance not mainly by recovering the

original node features, but by making the resulting node representations largely reflect graph topology.

- We further explore the true reasons for the utility drop under low privacy budgets. It turns out that the core component of various LDP mechanisms (*i.e.*, the unbiased rectification operation) introduces a large correction multiplier, which amplifies the magnitude of the inputs fed to the subsequent GNN component.
- We apply our new insights to reinterpret the state-of-the-art LDPGNN method UPGNET. We demonstrate that the utility gains primarily come from improved topology exploitation by HOA layer and the control of input scale by NFR layer, rather than the mechanisms emphasized in the original explanation.

Roadmap. The remainder of this paper is organized as follows: Section 2 presents the necessary preliminaries. In Section 3, we introduce the problem formulation and existing LDPGNN pipelines. Then, we propose our constructed feature-free baseline and reevaluate the representative LDPGNN methods in Section 4. Next, we provide a deep re-analysis for LDPGNN in Section 5. We further provide the discussion for future research in Section 6. We also discuss the related work in Section 7 and make the conclusion in Section 8.

2. Preliminaries

2.1. Local Differential Privacy

Local differential privacy (LDP) is a widely studied privacy-preserving approach used to collect user data with quantitative and controllable privacy loss [17], [18]. Its core idea is to make the output distributions statistically similar for any two different inputs, preventing the attacker from inferring the raw data. LDP protocols typically consist of two steps [9], [10], [11]. First, the data owners perturb their data locally and send the noisy reports to the server. Then, the server aggregates all the noisy reports to estimate the target query. Formally, we can define LDP as follows:

Definition 1 (ϵ -Local Differential Privacy). Let $\mathcal{D}$ denote the domain of a user’s private data and $\mathcal{O}$ denote the output domain of a randomized mechanism. A mechanism $\mathcal{M} : \mathcal{D} \rightarrow \mathcal{O}$ satisfies ϵ -local differential privacy if and only if for any pair of inputs $x, x' \in \mathcal{D}$ and any possible output $y \in \mathcal{O}$, we have

$$\Pr[\mathcal{M}(x) = y] \leq e^\epsilon \Pr[\mathcal{M}(x') = y], \quad (1)$$

where ϵ denotes the privacy budget, which balances the data utility and privacy protection. A smaller ϵ provides a stronger privacy guarantee but lower utility.

2.2. Graph Neural Networks

Graph neural networks (GNNs) are designed for learning representations on graph-structured data through the stacked graph convolution layers [1], [19], [20], [21],

TABLE 1: Main notations used in this paper.

Symbol	Description
ε	Privacy budget.
A	Adjacency matrix.
$\mathcal{N}(v)$	Number of neighbours of node v .
X	Feature matrix.
x_v	Feature vector of node v .
m	Feature-sampling parameter.
d	Feature dimension.
$\beta_{\mathcal{M}}(\varepsilon, d)$	Rectification coefficient.
K	Number of linear propagation steps.
Q	Linear propagation operator.
R	Random feature matrix.
d_r	Dimension of random features.
τ	The node feature regularization parameter.
$\alpha_{\varepsilon, m}$	Coefficient of the topology-only term.
Δ_{ε}	Zero-mean LDP-noise residuals.

[22]. Specifically, each layer aggregates information from a node’s local neighborhood and updates the node’s representation via a non-linear transformation. Formally, let $G = (V, A, X, Y)$ denote an undirected graph, where $V = \{v_1, v_2, \dots, v_n\}$ is the set of n nodes, and $A \in \{0, 1\}^{n \times n}$ is the adjacency matrix. $X \in \mathbb{R}^{n \times d}$ is the node feature matrix, where each node has a d -dimensional feature vector, and $Y \in \{0, 1\}^{n \times c}$ is the label matrix of c classes. An L -layer GNN consists of L graph convolution layers. At layer l , the representation h_v^l of node $v \in V$ is computed by aggregating the representations of its neighbors from the previous layer. This process can be formulated as:

$$h_{\mathcal{N}(v)}^l = \text{AGG}^l \left(\{h_u^{l-1} : u \in \mathcal{N}(v)\} \right), \quad (2)$$

$$h_v^l = \text{UPDATE}^l \left(h_{\mathcal{N}(v)}^l \right), \quad (3)$$

where $\mathcal{N}(v)$ denotes the set of neighbors of v , which may include v itself via a self-loop, and h_u^{l-1} is the embedding of node u at layer $l-1$. AGG^l is a differentiable, permutation-invariant aggregation function, such as mean, sum or max. Additionally, UPDATE^l is a trainable non-linear function, such as a multi-layer perceptron (MLP). The process is initialized with $h_v^0 = x_v$, i.e., the initial representation of node v is its feature x_v .

3. Problem Formulation and Existing Methods

3.1. Problem Formulation

In this paper, we focus on the semi-supervised node classification task, which aims to assign correct labels to unlabeled nodes. Formally, let $V = V_l \cup V_u$, where V_l and V_u denote the disjoint sets of labeled and unlabeled nodes, respectively. Each labeled node $v \in V_l$ is associated with a one-hot label vector, whose i -th entry is 1 if the node belongs to class i , and 0 otherwise. The training objective is to learn a classifier $f : V_l \rightarrow Y$ from the labeled nodes, and apply it to infer labels for nodes in V_u . We also

TABLE 2: Comparison of typical LDPGNN pipelines.

Method \ Step	Perturbation	Rectification	Denoising
LPGNN	MBM		Kprop
PrivGE	HDS	Calibrator	Kprop
UPGNET	MBM/PM		NFR + HOA

summarize the commonly used notations in this paper, as shown in Table 1.

Following previous LDPGNN works [12], [14], [16], we consider that the server only has access to the adjacency matrix A and the label matrix Y , while the feature matrix X is private. This privacy setting is practical in real-world scenarios, such as social networks [23], recommendation systems [24], or mobile services [25], where the platform may already maintain the interaction graph but should not directly access sensitive user attributes, such as user profiles, private preferences, and installed applications.

Our goal is to train a GNN model for node classification without directly revealing private node features. To this end, prior studies have adopted locally differentially private graph neural networks (LDPGNN), where users locally perturb their features under LDP, and the server rectifies and denoises the collected noisy reports before training the downstream GNN [12], [13], [14], [16].

3.2. Existing LDPGNN Methods

Existing studies on LDPGNNs typically follow a three-stage data processing pipeline: 1) Perturbation, 2) Rectification, 3) Denoising.

Perturbation. Each user applies an LDP mechanism $\mathcal{M}$ to locally randomize their d -dimensional feature vector x_v . Specifically, the user first samples m dimensions uniformly from the d dimensions without replacement. The selected m dimensions are then perturbed under a privacy budget of ε/m , while the remaining dimensions are padded with zeros. We denote the perturbed feature vector by x_v^* .

Rectification. The noisy report x_v^* received by the server is generally biased. Specifically, the expectation of x_v^* can be written as $\mathbb{E}[x_v^*] = a_{\mathcal{M}}x_v + b_{\mathcal{M}}$, where $a_{\mathcal{M}}$ and $b_{\mathcal{M}}$ are mechanism-specific coefficients determined by the privacy budget ε , the sampling parameter m , and the selected LDP mechanism $\mathcal{M}$. To obtain an unbiased estimate of x_v , the server rectifies the noisy report as $x_v' = \frac{x_v^* - b_{\mathcal{M}}}{a_{\mathcal{M}}}$.

Denoising. The server further reduces the perturbation noise by propagating the rectified features over the graph. Formally, let $Q \in \mathbb{R}^{n \times n}$ denote the linear propagation operator derived from the adjacency matrix A . The denoised feature matrix can be written as $\tilde{X} = QX'$, which serves as input to downstream training.

We then describe how existing methods instantiate these three stages, with an overview provided in Table 2.

LPGNN [12]. It protects node privacy by protecting two orthogonal aspects: features and labels. In this paper, we focus only on its feature protection mechanism. In the perturbation stage, each node locally perturbs its feature vector using the multi-bit mechanism (MBM). In the rectification and denoising stages, the server applies an MBM-specific unbiased rectifier and a K -hop propagation operator (Kprop) to mitigate the effect of injected noise.

PrivGE [26]. It is originally designed for privacy-preserving graph embedding rather than downstream GNN-based node classification. Its high-dimensional square-wave (HDS) mechanism extends square wave [27] to high-dimensional feature data and serves as its core feature perturbation mechanism. To adapt PrivGE to our unified LDPGNN evaluation pipeline, we combine HDS and its corresponding rectifier with the Kprop denoiser and downstream GNN backbones.

UPGNET [16]. It perturbs node features using MBM and the piecewise mechanism (PM) [11], [14], which we denote as UPGNET-MBM and UPGNET-PM, respectively. Furthermore, UPGNET improves LDPGNN utility by targeting two factors that affect feature estimation: the effective feature dimension and the neighborhood size. Specifically, in the denoising stage, UPGNET introduces node feature regularization (NFR) to reduce the effective feature dimension and high-order aggregation (HOA) to enlarge the effective neighborhood while mitigating oversmoothing.

3.3. Feature-free Baseline

LDPGNNs are motivated by the expectation that, when raw node features cannot be directly collected, their perturbed reports still provide useful information for downstream GNN training. However, in our setting, the graph topology is already available to the server, and only node features are protected. Thus, the key question is not merely whether perturbed features can support non-trivial GNN model training, but whether they provide utility beyond what the server can already obtain from the topology alone. This motivates a conservative baseline: instead of collecting perturbed node features, the server trains a GNN using only the observed graph topology.

This *feature-free baseline* tests whether LDP-based feature collection provides additional utility. Therefore, even if an LDPGNN achieves high accuracy or outperforms other LDPGNN methods, it does not demonstrate additional downstream value from collecting perturbed features unless it yields stable gains over the feature-free baseline. Next, we describe how we construct a simple feature-free baseline, against which we comprehensively reevaluate representative LDPGNN methods.

4. LDPGNN Reevaluation

4.1. Baseline Construction

By definition, LDP reports are randomized but still correlated with the original features. To construct a stricter

Algorithm 1: Construction of FeatFree

Input: Adjacency matrix $A \in \mathbb{R}^{n \times n}$; random feature dimension d_r ; propagation depth K

Output: Feature-free node representation matrix $X_{\text{FeatFree}} \in \mathbb{R}^{n \times d_r}$

- 1: **Step 1: Random feature generation**
 - 2: Sample $R \in \mathbb{R}^{n \times d_r}$ with $R_{ij} \stackrel{\text{i.i.d.}}{\sim} \mathcal{N}(0, 1)$
 - 3: **Step 2: Propagation operator construction**
 - 4: Compute the degree matrix D_A
 - 5: $(D_A)_{ii} = \sum_{j=1}^n A_{ij}$
 - 6: Compute the normalized adjacency matrix $\tilde{A}$
 - 7: $\tilde{A} = D_A^{-\frac{1}{2}} A D_A^{-\frac{1}{2}}$, $(D_A)_{ii}^{-1/2} = 0$ if $(D_A)_{ii} = 0$
 - 8: Set $P^{(0)} = I_n$
 - 9: **for** $k = 1, \dots, K$ **do**
 - 10: $P^{(k)} = \tilde{A} P^{(k-1)}$
 - 11: **end for**
 - 12: $Q = \frac{1}{K+1} \sum_{k=0}^K P^{(k)}$
 - 13: **Step 3: Feature-free representation**
 - 14: $X_{\text{FeatFree}} = QR$
 - 15: **return** X_{FeatFree}
-

feature-free baseline, we instead generate node features from random noise independent of all raw node features, which is equivalent to LDPGNN with a privacy budget of 0.

We refer to our constructed feature-free baseline as FeatFree. The specific baseline construction is summarized in Algorithm 1. First, we sample a random feature matrix $R \in \mathbb{R}^{n \times d_r}$ from a Gaussian distribution. Since R is independent of raw node features, it serves as a feature-free substitute for LDP reports. Next, we construct a linear propagation operator, which is motivated by message passing that the propagation can embed topological information into node representations. Specifically, we compute the degree matrix and normalized adjacency matrix $\tilde{A}$ (Line 4 - Line 7), generate the propagation matrices $\{P^{(k)}\}_{k=0}^K$ (Line 8 - Line 11), and average them to obtain the propagation operator $Q = \frac{1}{K+1} \sum_{k=0}^K P^{(k)}$ (Line 12). Finally, we obtain the FeatFree representations as $X_{\text{FeatFree}} = QR$, which successfully embed the graph structure information. The resulting matrix X_{FeatFree} is then used as the input feature matrix for the downstream GNN.

4.2. Experimental Setup

Datasets. As shown in Table 3, we conduct experiments on four representative real-world datasets, which are widely used in previous works [12], [16]. The description of the datasets is as follows:

- *Cora* [28] and *CiteSeer* [28] are citation networks, where each node represents a document and edges denote citation relationships. Each node has a bag-of-words feature vector and a label indicating its category.

TABLE 3: Statistics of datasets.

Dataset	Class	Node	Edge	Feature	Avg.Deg
Cora	7	2,708	5,278	1,433	3.90
Citeseer	6	3,327	4,552	3,703	2.74
LastFM	10	7,083	25,814	7,842	7.29
Facebook	4	22,470	170,912	4,714	15.21

- *LastFM* [29] is a social network collected from the music streaming service LastFM. Nodes represent users from Asian countries, and edges correspond to friendships. The task is to predict the home country of a user based on the artists liked by the user. Since the original dataset is highly imbalanced, we follow prior work [12] and keep only the ten classes with the largest number of samples.
- *Facebook* [30] is a social network, where nodes represent official Facebook pages, and edges represent mutual liking relationships. Each node has a feature extracted from the page description and a label indicating its category.

Competitors. We compare the feature-free baseline with four representative LDPGNN methods introduced in Section 3.2: LPGNN [12], PrivGE [26], UPGNET-MBM [16], and UPGNET-PM [16]. For a fair comparison, we follow the hyperparameter settings recommended in the original papers. For UPGNET, we adopt its stronger N-H architecture, where node feature regularization (NFR) is applied before high-order aggregation (HOA), and evaluate it with both perturbation mechanisms used in the original paper, including MBM and PM. Since PrivGE is originally designed for graph embedding, we adapt it to node classification by combining its HDS perturbation mechanism with the Kprop denoiser and the same downstream GNN backbones.

Parameter Settings. For all datasets, we randomly split nodes into training, validation, and test sets with a ratio of 2:1:1. To demonstrate the performance of our proposed feature-free baselines, we apply them with three state-of-the-art GNN architectures, including GCN [20], GraphSAGE [1] and GAT [21]. Due to space limitations, we defer the details of the parameters to Appendix B.

Implementation. We conduct experiments on the classic node classification task and report test accuracy as the primary metric. We implement with Python 3.10. All experiments are conducted on a server equipped with dual Intel Xeon Platinum 8488C CPUs, 512 GB of memory, and 4 NVIDIA A100 GPUs. Our implementation is available at https://anonymous.4open.science/r/Revisiting_LDPGNN-D18A.

4.3. Experimental Results

We vary the privacy budget ϵ from 0.001 to 10.0 to evaluate the performance of representative LDPGNN methods. Our goal is to examine whether these methods can achieve stable utility gains over FeatFree. Figure 1 presents the experimental results on four various datasets and three classic GNN backbones.

Surprisingly, we observe that across the evaluated settings, LDPGNN methods can hardly outperform FeatFree. Although small positive gaps occasionally appear, the maximum accuracy gain over FeatFree is only 0.79%, and the gain is negative in most cases. These small and unstable gaps do not constitute meaningful utility improvements. Specifically, the results fall into two regimes:

- *Low privacy budget regime* ($\epsilon \in [0.001, 1.0]$). In this range, LDPGNN methods suffer poor utility, and their performance further decreases as the privacy budget becomes smaller. FeatFree consistently outperform all LDPGNN methods.
- *Medium-to-high privacy budget regime* ($\epsilon \in [2.0, 10.0]$). In this range, the performance of LDPGNN methods plateaus. Increasing the privacy budget does not lead to substantial utility gains, and these methods still fail to consistently outperform the FeatFree.

In summary, these results suggest that, in the evaluated settings, collecting features under LDP does not provide a stable utility gain over using graph topology alone.

5. LDPGNN Re-analysis

5.1. Overview

As shown in the previous section, existing LDPGNN methods rarely provide any utility gain over FeatFree. This counterintuitive phenomenon raises two questions about where their utility comes from and why it still degrades under low privacy budgets. In this section, we answer these questions and further use the resulting insights to reinterpret the effectiveness of UPGNET, the current state-of-the-art LDPGNN method.

Q1: Why do perturbed features fail to provide additional utility over FeatFree? Intuitively, even though local perturbation injects noise into node features, the collected reports remain statistically correlated with the original features. Therefore, after rectification and graph propagation, they are expected to provide additional feature-dependent information beyond what is already available from the topology. However, the results in Section 4.3 show that such additional information does not convert into consistent downstream utility gains.

To answer this, we follow representation-similarity analysis, which characterizes representations through pairwise similarities or Gram matrices [31], [32], [33], [34]. Specifically, we introduce the Gram matrix (*i.e.*, the inner-product matrix) of node representations as an analytical tool to trace the sources of discriminative information. We theoretically show that, after linear graph propagation in the LDPGNN pipeline, the inner-product matrix of the representations admits a three-term decomposition consisting of a topology-only term, a raw-feature-related term, and an LDP-noise term. The key finding is that the topology-only term dominates the decomposition, indicating that the information in the representations is mainly induced by graph topology. To further examine whether the non-dominant raw-feature

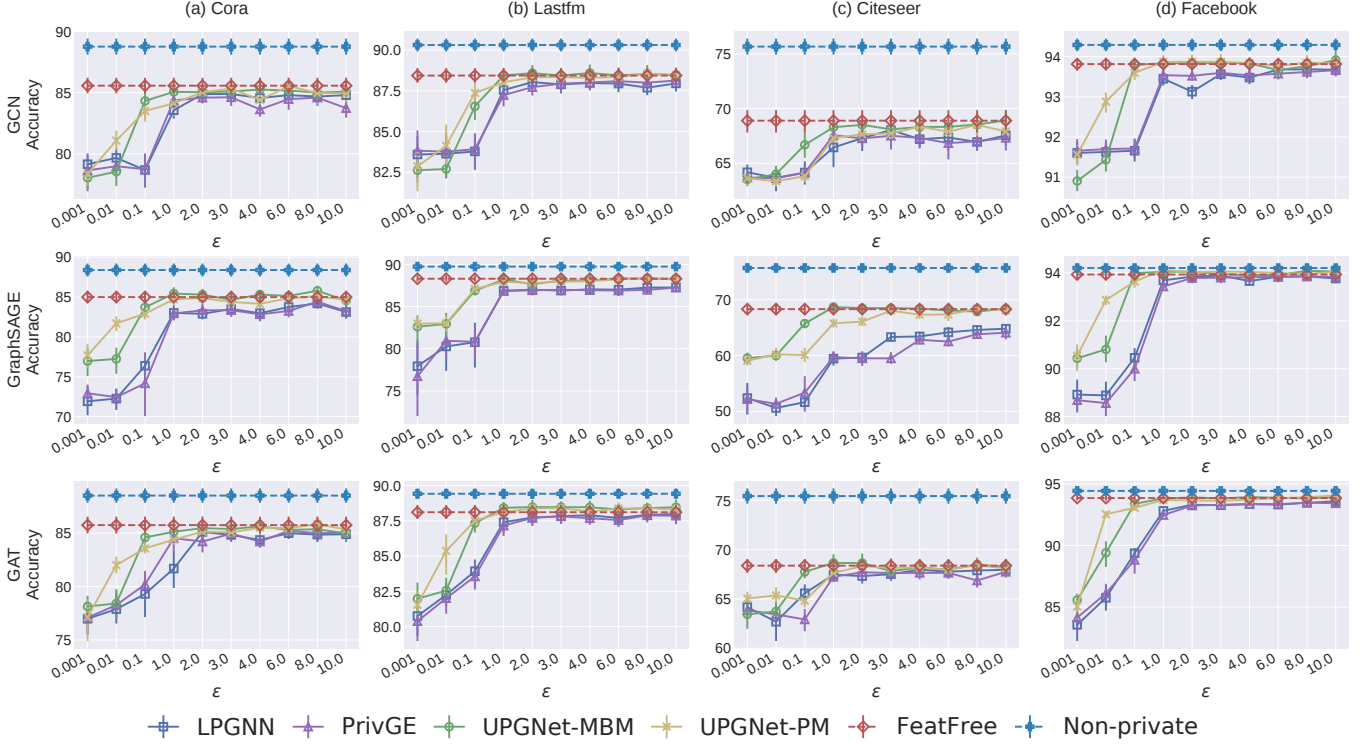


Figure 1: Performance of various methods. In each plot, x -axis represents the privacy budget ϵ and y -axis represents test accuracy (%). FeatFree represents feature-free baselines. Non-private represents a privacy-free reference without any applied LDP perturbation.

term or LDP-noise term contributes to utility, we conduct two ablation studies that progressively remove these two terms. The results show that the LDPGNN performance is largely preserved after these terms are removed, indicating that its utility is mainly driven by topology-derived information. Consequently, we conclude that the denoising stage in LDPGNNs does not primarily improve utility by recovering raw features. Instead, it mainly helps to embed topological information into node representations, which FeatFree already does without collecting feature reports. Meanwhile, the additional raw-feature-related information carried by LDP reports is weak and can be further obscured under the low privacy budgets. As a result, it does not provide stable utility gains for downstream classification. Overall, the dominant topological information and weak contribution of LDP reports explain why LDPGNNs fail to consistently outperform the feature-free baseline. We introduce the inner-product analysis object in Section 5.2 and present the theoretical decomposition results and ablation studies in Section 5.3.

Q2: Given that the information mainly comes from the graph topology, why do LDPGNNs still suffer severe utility degradation under low privacy budgets? If the downstream utility mainly comes from topology-derived information, then decreasing the privacy budget ϵ should not directly remove the primary source of information. However, in Section 4.3, the empirical results show that LDPGNNs

still suffer severe utility degradation in the low privacy budget regime.

We trace this degradation to the unbiased rectification of LDP mechanisms. Under representative LDP mechanisms, unbiased rectification amplifies the perturbed reports by a privacy-budget-dependent coefficient whose magnitude grows as $O(d/\epsilon)$. Although this numerical amplification does not change the main source of discriminative information, it pushes the representations fed into the downstream GNN into an excessively large numerical range, making the downstream model harder to train effectively on the topology-derived information. Additionally, the rectification multiplier becomes larger as ϵ decreases, making this numerical scale mismatch more severe and leading to a stronger degradation in trainability and classification utility. We further show that the low-budget utility drop can be largely mitigated by removing the rectification-induced numerical amplification. The detailed rectification analysis and empirical validation are provided in Section 5.4.

Reinterpret State-of-the-Art. Finally, we apply our theoretical framework to reinterpret the effectiveness of UPGNET, the current state-of-the-art LDPGNN method. We show that its high-order aggregation (HOA) mainly improves utility by allowing LDP reports to better exploit graph topology. Additionally, we show that its node feature regularization (NFR) improves utility by reducing the numerical amplification introduced by unbiased rectification. While these

combined mechanisms help the LDPGNN pipeline approach the performance of the feature-free baseline, they still fail to surpass it. We conclude the section with two targeted experiments that validate our reinterpretation of UPGNET. The detailed reinterpretations of HOA and NFR, together with empirical validations, are presented in [Section 5.5](#).

5.2. Analysis Object

The key to answering Q1 is to identify whether the collected LDP reports provide feature-dependent information beyond the topology-derived information already available to the feature-free baseline. However, the denoising stage in LDPGNNs embeds topological information into the node representations, which makes it difficult to directly separate raw feature information from topology-derived information.

Inspired by representation-similarity analysis [31], [32], [33], [34], [35], [36], we alternatively evaluate the source of information in node representations through pairwise similarities, allowing us to analyze how information is encoded in representations. Intuitively, if two nodes exhibit similar representations, the classifier is more likely to assign them similar predictions. Therefore, the pairwise similarities provide a compact description of the information available to the downstream classifier. A simple way to characterize such similarities is by calculating the inner product of the representation matrix. More importantly, if the inner product can be decomposed into distinct components, such as one component that depends on raw features and others independent of them, then we can trace which information sources contribute to the learned representations. This decomposition also enables ablation studies that isolate the contribution of each component.

Formally, let $\hat{X} \in \mathbb{R}^{n \times d}$ denote the representation matrix immediately before the downstream GNN, where $\hat{x}_v$ is the representation vector of node v . We define the inner-product matrix of $\hat{X}$ as $\hat{X}\hat{X}^\top$. The entry $\hat{x}_u^\top \hat{x}_v$ measures the similarity between the representations of nodes u and v . We refer to $\hat{X}\hat{X}^\top$ as the inner-product matrix throughout the remainder of this paper.

5.3. Answer to Question 1

In this section, we analyze the representations produced by the LDPGNN denoising stage to determine whether the collected LDP reports provide usable feature-dependent information beyond what FeatFree already obtains from graph topology. We use the multi-bit mechanism (MBM) [12] as an example for the following analysis.

Theorem 1. *Let $Q \in \mathbb{R}^{n \times n}$ denote the propagation operator, $X \in \mathbb{R}^{n \times d}$ denote the raw feature matrix, and $\hat{X} \in \mathbb{R}^{n \times d}$ denote the representation matrix after rectification and propagation under MBM. Then the inner-product matrix of $\hat{X}$ can be decomposed as*

$$\begin{aligned} \hat{X}\hat{X}^\top = & \underbrace{\alpha_{\varepsilon,m} QQ^\top}_{\text{topology-only term}} + \underbrace{Q(XX^\top - \text{Diag}(XX^\top))Q^\top}_{\text{raw-feature-related term}} \\ & + \underbrace{Q\Delta_\varepsilon Q^\top}_{\text{LDP-noise term}}, \end{aligned} \quad (4)$$

where $\alpha_{\varepsilon,m} = \frac{d^2(e^{\varepsilon/m} + 1)^2}{m(e^{\varepsilon/m} - 1)^2}$, and $\Delta_\varepsilon = H(X'X'^\top) - (XX^\top - \text{Diag}(XX^\top))$. Moreover, $\mathbb{E}[\Delta_\varepsilon | X] = 0$. The matrix $\text{Diag}(XX^\top)$ denotes the diagonal matrix formed by keeping only the diagonal entries of $XX^\top$, while the matrix $H(XX^\top)$ denotes the matrix formed by keeping only the off-diagonal entries of $XX^\top$.

Due to space constraints, the detailed proof is deferred to [Appendix A.1](#). [Theorem 1](#) decomposes the inner-product matrix of propagated LDP representation into three components. The first term, $\alpha_{\varepsilon,m} QQ^\top$, is topology-only. The matrix $QQ^\top$ is fully determined by the propagation operator Q , while the coefficient $\alpha_{\varepsilon,m}$ is independent of the raw node feature. Therefore, the first term contains no protected node feature information. The second term, $Q(XX^\top - \text{Diag}(XX^\top))Q^\top$, is raw-feature-related. It captures how correlations among raw node features are propagated through the graph. The third term, $Q\Delta_\varepsilon Q^\top$, whose expectation is zero, is an LDP-noise term, introduced by the randomness of the LDP mechanism.

The key observation lies in the coefficient of the topology-only term. For fixed m , when using MBM as the perturbation mechanism, the coefficient satisfies

$$\alpha_{\varepsilon,m} = \frac{d^2(e^{\varepsilon/m} + 1)^2}{m(e^{\varepsilon/m} - 1)^2} = \Theta\left(\frac{d^2}{\varepsilon^2}\right) \quad \text{as } \varepsilon/m \rightarrow 0. \quad (5)$$

Thus, for high-dimensional node features and moderate privacy budgets, the topology-only term can be numerically large. For example, as shown in [Figure 2](#), on Cora, the coefficient remains 7.13×10^5 even when $\varepsilon = 10$.

In contrast, as shown in [Equation 4](#), the raw-feature-related term is not amplified by this coefficient. Moreover, real-world graph datasets often use sparse and normalized input features, which further limits the scale of this term. Thus, although this raw-feature-related component is the only term that can carry additional feature-dependent information, its contribution is small compared with the topology-only component and can be further obscured by the LDP-noise term. Consequently, $\alpha_{\varepsilon,m} QQ^\top$ becomes the dominant component in the inner-product matrix of the propagated representation.

This decomposition provides an initial explanation for the empirical failure observed in [Section 4.3](#). It suggests that the dominant information in LDPGNN representations is inherited from graph topology, while the additional feature-dependent information carried by LDP reports may be too weak to provide stable utility gains. To further verify that the remaining non-dominant components do not support the observed utility of the LDPGNN pipeline, we next remove the raw-feature-related component and the LDP-noise com-

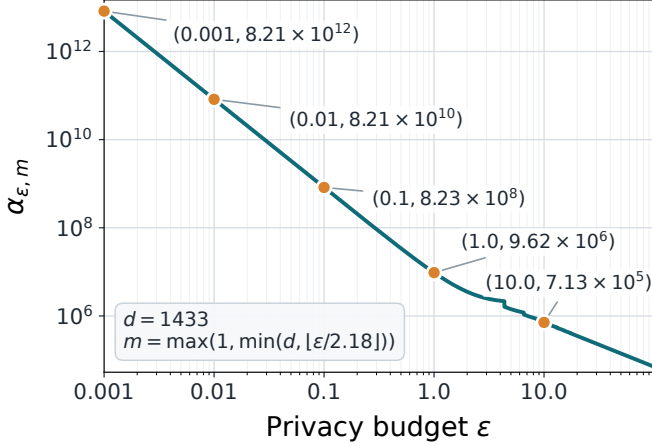


Figure 2: Variation of the coefficient $\alpha_{\epsilon, m}$ with the privacy budget ϵ . The curve is calculated using the configuration of the Cora dataset with MBM as the perturbation mechanism.

ponent in sequence and examine whether the observed utility can still be preserved.

Removing the Raw-feature-related Term. We first examine whether the utility of LDPGNNs depends on the raw-feature-related term. Specifically, we replace the LDP reports with simulated feature-independent reports, which follow the same mechanism format but are generated without using raw feature values. For MBM, the original report distribution depends on the raw feature value x through

$$\Pr[x^* | x] = \begin{cases} \frac{m}{2d} \left(1 + \frac{e^{\epsilon/m} - 1}{e^{\epsilon/m} + 1} x\right), & x^* = 1, \\ \frac{m}{2d} \left(1 - \frac{e^{\epsilon/m} - 1}{e^{\epsilon/m} + 1} x\right), & x^* = -1, \\ 1 - \frac{m}{d}, & x^* = 0. \end{cases}$$

In the simulated feature-independent version, we keep the same sampling probability and output distribution but remove the dependence on x :

$$\Pr[x^* | x] = \begin{cases} \frac{m}{2d}, & x^* = 1 / -1, \\ 1 - \frac{m}{d}, & x^* = 0. \end{cases}$$

Then, if we let $\hat{X}_{\text{SIM}}$ denote the propagated representation produced by this simulated pipeline, its inner-product matrix removes the raw-feature-related term and takes the form as follows:

$$\hat{X}_{\text{SIM}} \hat{X}_{\text{SIM}}^\top = \alpha_{\epsilon, m} Q Q^\top + Q \Delta_{\epsilon, \text{SIM}} Q^\top, \quad (6)$$

where $\Delta_{\epsilon, \text{SIM}}$ is a LDP-noise random variable whose expectation is zero.

Empirical Results. The results are shown in Figure 3. Across all representative LDP mechanisms, replacing real LDP reports with matched raw-feature-independent reports produces almost the same accuracy trend and level as the original LDP pipeline. At low privacy budgets, both LDP and SIM have low accuracy. As the privacy budget in-

TABLE 4: Performance comparison among the propagation-profile feature-free baseline (FeatFree-P) and the LDPGNN method with the highest test accuracy among all methods evaluated in Section 4.3 under $\epsilon = 10$ (Best-LDP). FeatFree-P uses HOA as its propagation operator.

Backbone	Setting	Cora	LastFM	CiteSeer	Facebook
GCN	Best-LDP	85.08	88.47	68.92	93.91
	FeatFree-P	85.86 $\uparrow 0.78$	88.68 $\uparrow 0.21$	69.52 $\uparrow 0.60$	94.18 $\uparrow 0.27$
SAGE	Best-LDP	84.62	88.27	68.43	94.06
	FeatFree-P	85.72 $\uparrow 1.10$	88.41 $\uparrow 0.14$	69.57 $\uparrow 1.14$	94.44 $\uparrow 0.38$
GAT	Best-LDP	85.33	88.47	68.22	94.06
	FeatFree-P	85.91 $\uparrow 0.58$	88.32 $\downarrow 0.15$	69.64 $\uparrow 1.42$	93.96 $\downarrow 0.10$

creases from 0.1 to 2, both performance improves rapidly. Under larger privacy budgets, both curves plateau. More importantly, SIM is not consistently worse than LDP under the same privacy budget. In several cases, SIM even exceeds the LDP curve, such as in the low-budget and high-budget regimes of MBM and the high-budget regime of PM. The result indicates that the accuracy of the original LDPGNN pipeline can be largely reproduced without using raw-feature-dependent information.

Further Removing the LDP-noise Term. We next further examine whether the remaining noisy term is unnecessary for the observed utility. Since the leading topology-only term in Theorem 1 is proportional to $Q Q^\top$, a direct way to isolate this component is to use the propagation operator Q itself as the node representation. Each row of Q can be viewed as a propagation profile that describes how a node aggregates information from all other nodes under the chosen propagation operator. Therefore, unlike FeatFree, which propagates random input features through Q , this construction directly uses the topology information as feature-free node representations. Specifically, for node v , it directly uses the v -th row of the propagation operator Q to rewrite the feature of node v and sends this representation to GNNs for training. If we let $X_{\text{Profile}} = Q$, then

$$X_{\text{Profile}} X_{\text{Profile}}^\top = Q Q^\top. \quad (7)$$

This construction removes both the raw-feature-related term and the LDP-noise term, leaving only the topology-derived information induced by the propagation operator. We refer to it as the propagation-profile feature-free baseline. To distinguish it from the feature-free baseline we constructed earlier in Section 4.1, we denoted the propagation-profile feature-free baseline by FeatFree-P. If FeatFree-P matches or outperforms LDPGNNs, then removing both the raw-feature-related term and the LDP-noise term does not reduce the observed utility. This indicates that the topology-only structure is sufficient to explain LDPGNN performance.

Empirical Results. The results are shown in Table 4. We compare FeatFree-P with the best-performing LDPGNN method at $\epsilon = 10$. FeatFree-P achieves comparable, and often slightly better, performance than the best LDPGNN method. This result shows that a representation constructed directly from the propagation operator is already sufficient

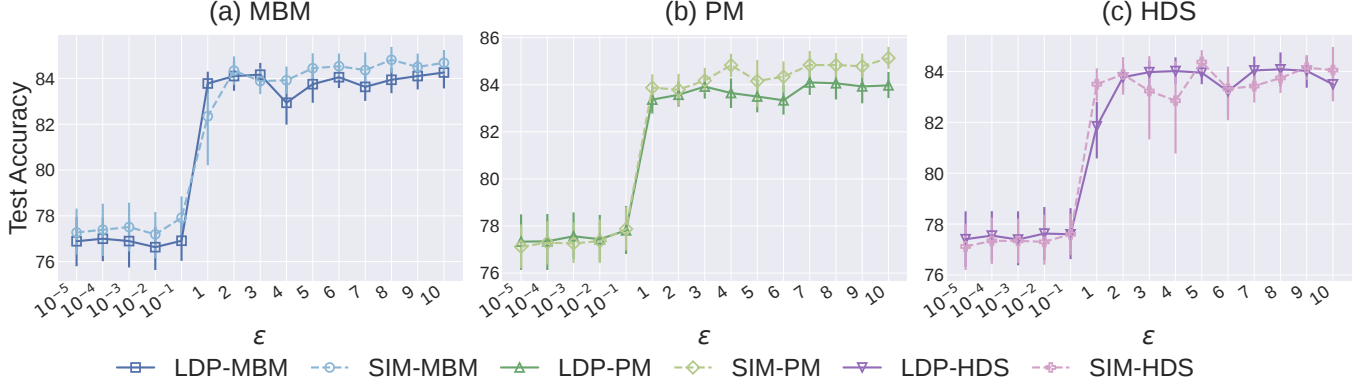


Figure 3: Performance comparison between original and simulated LDPGNN methods. All methods apply HOA for denoising and use GraphSAGE as the backbone. The evaluation covers three perturbation mechanisms (MBM, PM, and HDS) against their simulated equivalents. The “LDP-” prefix designates the original LDPGNN pipelines, while the “SIM-” prefix indicates the simulated ones.

to match existing LDPGNNs. Therefore, both the LDP-noise and raw-feature-related components is unnecessary for the observed LDPGNN utility.

Putting Things Together. The decomposition and the two ablation studies provide a unified answer to Q1. In terms of downstream utility, the denoising stage in LDPGNNs primarily works by embedding topological information into node representations through linear graph propagation. Since feature-free baselines also construct representations from similar topological information and the LDP-reports hardly provide additional usable information, LDPGNNs naturally fail to achieve consistent gains over feature-free baselines.

5.4. Answer to Question 2

According to the source decomposition in [Theorem 1](#) and the coefficient analysis in [Equation 5](#), as ε decreases, the coefficient $\alpha_{\varepsilon, m}$ grows and makes the topology-only component $\alpha_{\varepsilon, m} Q Q^T$ increasingly dominant relative to the raw-feature-related component. Since this dominant information source is fully determined by the non-private graph topology, one might expect the downstream utility to remain stable, or at least not drop merely because the privacy budget becomes smaller. However, the empirical results in [Figure 1](#) show the opposite: LDPGNNs suffer substantial utility degradation in the low-budget regime.

We compare how topology-only baselines and LDPGNN methods construct their input representations. The key difference is the unbiased rectification step. Specifically, LDPGNNs rectify the locally perturbed reports before graph propagation, whereas topology-only baselines do not. This step is directly controlled by the privacy budget, as it amplifies the perturbed reports by a multiplier before propagation. Therefore, rectification provides a natural lens for understanding why LDPGNNs suffer utility degradation under low privacy budgets.

Following prior implementations of MBM [\[12\]](#), PM [\[16\]](#), and HDS [\[26\]](#), we summarize their unbiased

TABLE 5: Rectification formulas for typical mechanisms.

Mech.	Optimal m	Rectification formula
MBM	$\max\{1, \min(d, \lfloor \varepsilon/2.18 \rfloor)\}$	$x'_v = \frac{d(e^{\varepsilon/m} + 1)}{m(e^{\varepsilon/m} - 1)} x_v^*$
PM	$\max\{1, \min(d, \lfloor \varepsilon/2.5 \rfloor)\}$	$x'_v = \frac{d(1 + e^{-\varepsilon/(2m)})}{m(1 - e^{-\varepsilon/(2m)})} x_v^*$
HDS	$\max\{1, \min(d, \lfloor \varepsilon/3.317 \rfloor)\}$	$x'_v = \frac{d}{m \left(1 - \frac{1 - e^{-\varepsilon/m}}{\varepsilon/m}\right)} x_v^*$

rectification formulas in [Table 5](#), where the optimal sampling choices are calculated by minimizing the variance of the estimator x'_v . We observe that if the original features are normalized to $[-1, 1]$, the unbiased rectification step in the LDP mechanism is mathematically equivalent to multiplying the LDP reports by a coefficient determined by the privacy budget ε , the feature dimension d , and the sampled dimension m . Furthermore, we theoretically analyze the order of growth of this rectification multiplier with respect to ε and d :

Theorem 2. For a mechanism $\mathcal{M} \in \{\text{MBM}, \text{PM}, \text{HDS}\}$, let $\beta_{\mathcal{M}}(\varepsilon, d)$ denote the coefficient used in the rectification formula in [Table 5](#). Let the sampled dimension $m_{\mathcal{M}}$ be chosen exactly as in [Table 5](#), with constants $t_{\text{MBM}} = 2.18$, $t_{\text{PM}} = 2.5$, and $t_{\text{HDS}} = 3.317$:

$$m_{\mathcal{M}} = \max \left(1, \min \left(d, \left\lfloor \frac{\varepsilon}{t_{\mathcal{M}}} \right\rfloor \right) \right).$$

In the privacy regime $0 < \varepsilon \leq t_{\mathcal{M}}d$, the rectification multiplier of each mechanism satisfies

$$\beta_{\mathcal{M}}(\varepsilon, d) = O \left(\frac{d}{\varepsilon} \right).$$

The detailed proof is deferred to [Appendix A.2](#). [Theorem 2](#) demonstrates that for MBM, PM, and HDS, this operation multiplies the LDP reports by a multiplier

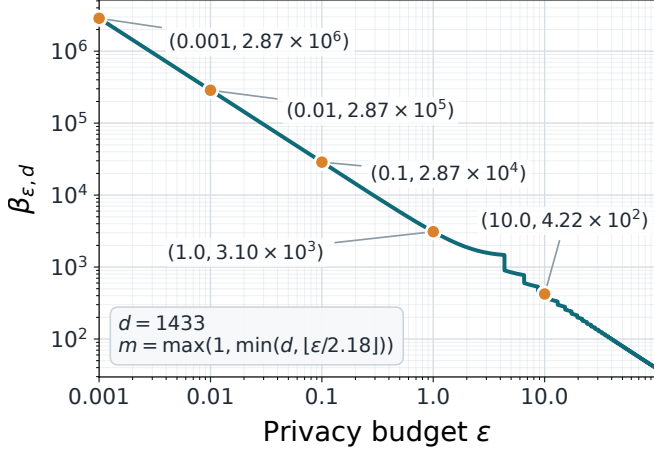


Figure 4: Variation of the coefficient $\beta_{\epsilon,d}$ with the privacy budget ϵ . The curve is calculated using the configuration of the Cora dataset with MBM as the perturbation mechanism.

that grows with $O(d/\epsilon)$ in the low-budget and moderate-budget regimes. Since graph datasets typically have high-dimensional node features, this multiplier can become numerically massive even under moderate privacy budgets. For instance, as illustrated in Figure 4, when applying MBM on the Cora dataset, the rectification multiplier remains as large as 4.22×10^2 even at $\epsilon = 10$.

Why does such Multiplier Matter? The large rectification multiplier does not merely guarantee statistically unbiased. It directly changes the magnitude of the node representations fed into the downstream GNN. This can affect optimization because neural network training is sensitive to the numerical scale of inputs, activations, and gradients. Prior work has shown that maintaining stable activations and gradients is important for effective optimization, while poor numerical scale control can hinder trainability [37], [38]. This is also a key motivation behind normalization techniques such as Batch Normalization [39], [40], which improve optimization by controlling the numerical scale of input features.

LDP rectification produces exactly such a numerical scale mismatch at the input of the downstream GNN. As shown in Theorem 2, the rectification multiplier grows as the privacy budget, causing the representations fed into the downstream GNN to have increasingly large numerical magnitudes. Although this rescaling does not remove the topology-derived information, it makes the input scale poorly matched to neural-network training, thereby impairing trainability and reducing utility. This explains why the model utility consistently drops as the privacy budget.

We validate this explanation through a controlled rescaling experiment. Specifically, we fix the perturbed LDP reports, the propagation operator, and the downstream GNN architecture, and vary only an additional divisor applied to the rectified representations before classification. Formally, given the original rectified representation matrix $\hat{X}$, we

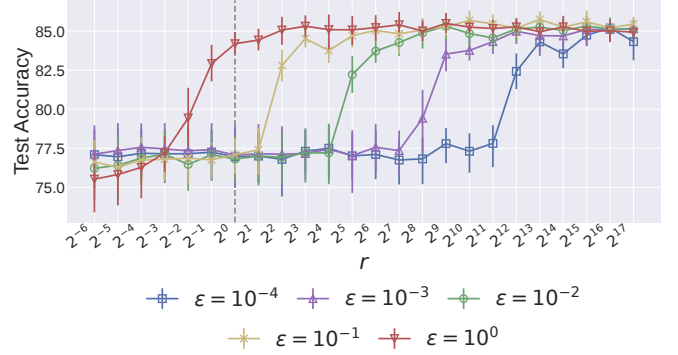


Figure 5: Impact of manual feature re-scaling divisor r on downstream GNN utility across varying privacy budgets. The vertical dashed line at $r = 2^0$ corresponds to the standard unbiased rectification without additional re-scaling. A larger r ($r > 1$) shrinks the feature scale, whereas a smaller r ($r < 1$) further amplifies it.

construct the actual GNN input as

$$\hat{X}^{(r)} = \frac{1}{r} \hat{X}, \quad (8)$$

where r is the divisor. By varying r , we can gradually remedy the input amplification while keeping all other components unchanged.

Empirical Results. As shown in Figure 5, scaling down the rectified representations substantially restores model utility. Moreover, since smaller privacy budgets induce larger rectification multipliers, they require larger scaling divisors to bring the input representations back to a numerically appropriate range. The results show that rectification-induced numerical amplification is a key factor of utility degradation.

Remark. Since Figure 5 shows that utility drop under low privacy budget can be easily remedied by removing the unbiased rectification operation, it reminds us of the necessity of rectification for neural-network-based downstream tasks, even though natural for traditional LDP estimation tasks. In classical LDP tasks such as frequency estimation, the server aims to recover an unbiased estimate of an absolute statistical quantity, so correcting the scale of the noisy reports is directly tied to the estimation objective. In contrast, a downstream GNN is more sensitive to the relative numerical size of features rather than their absolute numerical size. For a simple illustration, consider the first trainable layer $h = \sigma(Wx + b)$, where W is a learnable transformation matrix, b is a learnable bias vector, and σ is an activation function. If a rectified input is $x'_v = \frac{x_v^* - b_{\mathcal{M}}}{a_{\mathcal{M}}}$, then

$$Wx'_v + b = \underbrace{\frac{1}{a_{\mathcal{M}}} W}_{W'} x_v^* + \underbrace{\left(b - \frac{1}{a_{\mathcal{M}}} W b_{\mathcal{M}}\right)}_{b'}, \quad (9)$$

which means that the neural networks can absorb the statistical bias of the input features by adjusting the layer weight to W' and b' . By contrast, what matters more for neural-

network training is whether the input representations lie in a numerically suitable range for optimization as mentioned above. In all, for LDPGNNs, strict unbiased feature estimation can conflict with downstream trainability.

5.5. Reinterpretation of UPGNET

Original View. UPGNET introduces two modules in the denoising stage, namely high-order aggregation (HOA) and node feature regularization (NFR). Both modules are motivated by reducing the estimation error of the node feature. Specifically, for any node v and failure probability $\delta > 0$, the maximum coordinate-wise estimation error [16] after one linear aggregation is bounded as

$$\max_{i \in [d]} \xi_i = O\left(\frac{\sqrt{d \log(d/\delta)}}{\varepsilon \sqrt{|\mathcal{N}(v)|}}\right), \quad (10)$$

where d is the feature dimension, ε is the privacy budget, and $|\mathcal{N}(v)|$ denotes the neighborhood size used for aggregation. This bound suggests two directions for reducing the estimation error: increasing the effective neighborhood size and reducing the feature dimension. Following this intuition, HOA aggregates higher-order neighborhoods while mitigating over-smoothing, and NFR reduces the effective feature dimension by encouraging sparse node features.

However, the connection between these explanations and their actual working mechanism is incomplete on HOA and even wrong on NFR. For HOA, changing the aggregating way may more directly improve how graph topology is encoded into the propagated representations rather than better estimate raw node features. For NFR, the dimension d in Equation 10 refers to the original private feature dimension, whereas NFR sparsifies the server-side representations after rectification, which is targeted as reducing the number of non-zero dimensions. Since reducing non-zero dimensions can not affect the raw node feature dimension d , the explanation for how NFR works is wrong. This leaves room for a different interpretation of why NFR improves utility.

Reinterpretation. Our analysis indicates that their utility gains are better explained by the two insights identified respectively in Section 5.3 and Section 5.4. HOA and NFR should be reinterpreted accordingly: HOA improves how topology is encoded into node representations, while NFR mitigates the excessive numerical amplification introduced by unbiased rectification.

5.5.1 Reinterpreting HOA

HOA was originally introduced as an improved feature denoising operator. Based on Equation 10, the original motivation is that one-hop aggregation may involve too few neighbors to obtain a low-variance estimate of the raw feature on real-world graphs. A natural remedy is to apply more rounds of linear graph propagation so that each node can aggregate representation from a larger neighborhood. Meanwhile, using only the final result of deep propagation, as in Kprop, may make node representations over-smoothing, so HOA combines the representations from

TABLE 6: Performance comparison of the FeatFree that uses Kprop and HOA respectively as propagation operators.

Backbone	Setting	Cora	LastFM	CiteSeer	Facebook
GCN	Kprop	85.11	88.29	68.77	93.12
	HOA	85.45 $\uparrow 0.34$	88.43 $\uparrow 0.14$	69.07 $\uparrow 0.30$	93.35 $\uparrow 0.23$
SAGE	Kprop	84.86	88.17	67.49	93.13
	HOA	85.08 $\uparrow 0.22$	88.32 $\uparrow 0.15$	68.11 $\uparrow 0.62$	93.54 $\uparrow 0.41$
GAT	Kprop	85.17	88.05	68.41	93.26
	HOA	85.30 $\uparrow 0.13$	88.15 $\uparrow 0.10$	68.79 $\uparrow 0.38$	93.33 $\uparrow 0.07$

different propagation orders rather than keeping only the last one. Formally, let $\tilde{A}$ be the normalized adjacency matrix and let K be the propagation depth. The Kprop and HOA propagation operators can be written as

$$Q_{Kprop} = \tilde{A}^K, \quad Q_{HOA} = \frac{1}{K+1} \sum_{k=0}^K \tilde{A}^k. \quad (11)$$

Thus, Kprop uses only the K -hop propagation result, whereas HOA mixes propagation profiles from order 0 to order K .

However, according to Theorem 1, the dominant component in the propagated LDP representations is the topology-only term. Therefore, once the propagation operator is changed from Kprop to HOA, the dominant part is changed from $\alpha_{\varepsilon, m} Q_{Kprop} Q_{Kprop}^\top$ to $\alpha_{\varepsilon, m} Q_{HOA} Q_{HOA}^\top$, meaning that HOA changes how graph topology information is embedded into the representation. Intuitively, by retaining lower-order propagation terms while incorporating higher-order ones, HOA can encode broader structural information without completely discarding local structural distinctions.

Empirical Validation. To validate this reinterpretation, we compare the performance of feature-free baselines using different propagation operators, which can directly test whether changing the propagation operator can improve how representations exploit topological information, since feature-free baselines only construct the representation based on topology.

As shown in Table 6, using HOA as the linear propagation operator can improve the utility by better exploiting topological information relative to using Kprop as the linear propagation operator. Therefore, the effectiveness of HOA in UPGNET should be interpreted, at least in large part, as improved exploitation of graph topology rather than merely improved recovery of raw node features.

5.5.2 Reinterpreting NFR

NFR works by reducing the number of non-zero dimensions in the server-side representations on the rectified LDP reports. Specifically, the process of sparsifying the representation is to solve an optimization problem. Formally, for a node v , given its rectified report x'_v and denoting x''_v as the output of this module, NFR solves

$$x''_v = \arg \min_{w \in \mathbb{R}^d} \frac{1}{2} \|x'_v - w\|_2^2 + \mu \|w\|_1, \quad (12)$$

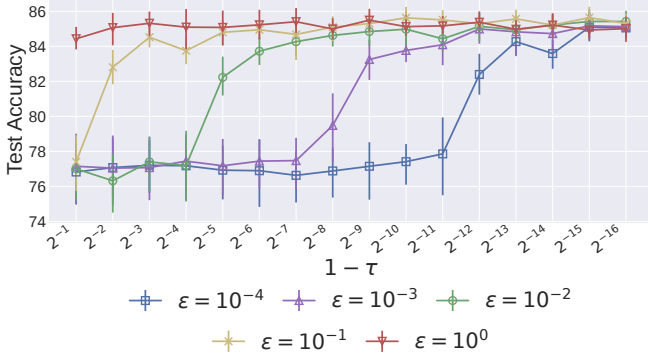


Figure 6: Sensitivity of classification accuracy to the NFR regularization strength parameter τ . The x -axis reports $1 - \tau = 2^x$, with x decreasing from -1 to -16 . Hence, moving right means τ approaches 1 and NFR applies stronger shrinkage.

where w is an auxiliary optimization variable vector over all possible regularized feature vectors. This objective has a closed solution

$$(x''_v) = \text{sign}((x'_v)) \cdot \max(|(x'_v)| - \mu, 0). \quad (13)$$

This closed form clarifies that NFR subtracts μ from the magnitude of the rectified representation, which reveals that NFR can act as an operator that mitigates the numerical amplification introduced in the rectification stage.

In UPGNET, when the NFR layer is placed before HOA, the regularization strength or, in our reinterpretation, the degree of mitigating is set as

$$\mu = \tau B, \quad (14)$$

where $\tau \in (0, 1)$ is a tunable hyperparameter and B denotes the radius of the symmetric range of rectified outputs. Consider MBM as an example to explain how the parameter τ works in NFR. According to Table 5, the rectified MBM report has coordinate values in $\{-\beta_{MBM}(\epsilon, d), 0, \beta_{MBM}(\epsilon, d)\}$, where $\beta_{MBM}(\epsilon, d) = \frac{d(e^{\epsilon/m} + 1)}{m(e^{\epsilon/m} - 1)}$. Thus, $B = \beta_{MBM}(\epsilon, d)$ and $\mu = \tau \beta_{MBM}(\epsilon, d)$. For a dimension of the rectified perturbed report with value B , the operation in Equation 13 reduces its value to $(1 - \tau)B$. Therefore, NFR substantially mitigates the rectification-induced numerical amplification before the representations are fed into HOA and the downstream GNN.

This reinterpretation also explains why the original UPGNET hyperparameter grid can not fully eliminate the low-budget utility drop as shown in Figure 1. When τ is searched only over $\{0.1, 0.3, 0.5, 0.7, 0.9\}$, even the strongest mitigating leaves a residual numerical value on the order of $0.1B$. When $B = O(d/\epsilon)$ is very large, this residual can still be too large for stable downstream training. Consequently, the model can still exhibit decreasing utility as the privacy budget becomes smaller, even though NFR already provides partial mitigation.

Empirical Validation. To validate this reinterpretation, we

extend the search range of τ beyond the grid $\{0.1, 0.3, 0.5, 0.7, 0.9\}$, used in the original UPGNET setting and move it much closer to 1. Since $\mu = \tau B$, increasing τ directly strengthens the degree of mitigation applied to the amplified features.

As shown in Figure 6, classification accuracy gradually recovers as τ approaches 1. Moreover, the recovery requires larger τ under smaller privacy budgets, because smaller ϵ introduces a larger rectification multiplier. This trend supports our reinterpretation: NFR improves utility primarily by reducing the numerical amplification on representation introduced by unbiased rectification. The above experimental results are similar to what we observed in Figure 5, where we mitigate the amplification by dividing the representation by a varying constant.

6. Discussion

The preceding analysis exposes two limitations in current LDPGNN research: a design mismatch and an evaluation gap. The design mismatch is that existing pipelines are commonly motivated as feature-estimation and denoising procedures, while their downstream utility is largely determined by how server-side exploits graph topology and whether the resulting representations remain trainable. The evaluation gap is that high downstream accuracy under feature-dependent LDP does not alone demonstrate that collecting private feature reports can provide additional utility. The reason is that public graph topology can already contribute to most of the high utility.

These observations suggest two directions for future LDPGNN research. First, server-side processing should be redesigned to extract feature information that is additional to topology while maintaining numerical stability. Second, feature collection should be evaluated by its marginal utility beyond topology, rather than by downstream accuracy alone.

Design Implication. Future LDPGNN methods may need to move beyond the standard “perturb–rectify–denoise” pipeline. Unlike traditional LDP tasks such as frequency estimation, server-side processing for LDPGNNs should not be optimized only for unbiased feature estimation. It should also ensure that the resulting representations remain well-scaled and trainable for downstream GNNs. Otherwise, rectification may be statistically unbiased but still produce poorly calibrated inputs, thereby degrading the downstream optimization. Additionally, future LDPGNN methods should first make better use of graph topology when constructing node representations, and then extract additional classification-relevant information from perturbed feature reports. In this way, private feature collection can provide additional utility beyond topology-only baselines.

Evaluation Implication. Based on this design perspective, a stronger future LDPGNN method should first demonstrate that collecting perturbed features provides additional downstream utility. A necessary way to verify this is to include a feature-free baseline as a reference point. Since such a baseline uses the same public topology but no private node

features, the utility gain over it provides an operational measure of whether the collected LDP reports contribute information beyond topology. An LDPGNN method should therefore report not only its absolute accuracy, but also its gain over the feature-free baseline. If this gain is not positive and statistically stable, the protocol introduces additional communication, computation, and privacy-budget costs without clear practical benefit.

7. Related Work

In the section, we discuss the related work from the following two aspects: broader LDPGNN and graph representation learning.

Broader LDPGNN. Locally differentially private graph learning can be studied under various privacy settings, including node-level and edge-level privacy, which protect the node features and the existence of edges, respectively. Our work focuses on the node-level setting, where graph topology is public while node features are private. Representative methods in this setting, including LPGNN [12], PrivGE [26], and UPGNET [16], locally perturb node features and then apply server-side rectification and propagation for downstream learning. Other work studies edge-level privacy, where the existence of edges or local neighborhoods is protected. For instance, LGA-PGNN [14] preserves user privacy by injecting local differential privacy noise into the decentralized local graphs held by different data holders. Moreover, this method performs local neighborhood augmentation on low-degree vertices to enhance the expressiveness of the learned model. In addition, a Topology-aware Differential Privacy Graph Neural Network learning framework (TDP-GNN) [41] is proposed to achieve personalized privacy protection for each node with improved privacy-utility guarantees. TDP-GNN identifies the topological importance of each node via an adjacency information entropy method. On this basis, the personalized topology-aware privacy budget is designed to quantify the privacy sensitivity of each node and adaptively allocate the privacy protection strength. Using link local differential privacy over decentralized nodes, BLINK [42] is designed to ensure the collaboration with an untrusted server to train GNNs without revealing the existence of any link. This method spends the privacy budget separately on links and degrees of the graph for the server to better denoise the graph topology based on Bayesian estimation [43], alleviating the negative impact of LDP on the accuracy of the trained GNNs. The above settings are complementary to ours, but they are not direct baselines for this work since our evaluation considers the public-topology and private-feature setting.

Graph Representation Learning. Our feature-free baseline is related to graph representation learning methods that extract predictive information from topology without relying on natural node attributes. Classical network embedding methods learn node representations directly from graph structure. DeepWalk [44] treats truncated random walks as word sequences and applies language-modeling

ideas to learn latent node representations. Furthermore, node2vec [45] extends this idea by using biased random walks to flexibly trade off breadth-first and depth-first neighborhood exploration, thereby capturing different notions of network neighborhoods. GraphWave [46] instead focuses on structural roles and uses heat wavelet diffusion patterns to embed nodes with similar local structural neighborhoods close to each other, even if they are far apart in the graph. Earlier link-based classification [47] shows that adjacency and link distributions can be directly predictive for node labels. More recently, LINKX [48] demonstrates that methods based on adjacency profiles can be strong on large non-homophilous graphs. These works support the observation that graph topology itself can contain substantial task-relevant information.

In addition, there are also some studies on artificial node features and random feature augmentation for GNN training. Cui *et al.* [49] distinguish positional and structural node features for non-attributed graphs and provide empirical guidance on when different artificial features are useful. Abboud *et al.* [50] point out that standard GNNs are limited in their expressive power. They further propose Random node initialization (RNI), which can effectively increase the expressive power of message-passing GNNs and can even yield universality results in expectation. Sato *et al.* [51] further show that adding random features can strengthen GNNs and help them solve the tasks that standard message-passing models cannot solve. Inspired by the power iteration method and its generalizations, Random Feature Propagation (RFP) [52] is proposed. This method connects random features with spectral positional encodings by propagating random node features through graph-dependent operators and retaining intermediate propagation results. Our work differs from these works above by using topology-derived representations not as independent graph embeddings, but as feature-free baselines for evaluating existing LDPGNNs under public topology.

8. Conclusion

In this paper, we revisit locally differentially private graph neural networks from a feature-collection perspective. We observe that all existing LDPGNN methods overlook a simple baseline that only relying graph topology without using node features. By reevaluating the representative LDPGNN methods, we find that these LDPGNN methods can hardly outperform our proposed feature-free baseline. Our theoretical and empirical analysis shows that the utility of existing LDPGNN pipelines mainly comes from topology-derived signals introduced by graph propagation, rather than from the recovery of private node features. We further identify unbiased rectification as a key cause of the utility drop under low privacy budgets, since it induces a severe numerical amplification. These insights also provide a new explanation for the state-of-the-art LDPGNN method UPGNET, whose gains are primarily attributed to better topology exploitation and scale control. Our findings call for a rethinking of LDPGNN design and evaluation.

Ethics Considerations

This paper studies the design and evaluation of locally differentially private graph neural networks (LDPGNNs). The work is motivated by privacy risks in raw node features and evaluates whether collecting LDP-perturbed features is necessary when graph topology is already available under the threat model used by prior LDPGNN work. The study is conducted entirely offline on standard benchmark graph datasets, including Cora, CiteSeer, LastFM, and Facebook. We do not recruit participants, interact with users, collect new data, query live systems, or attempt to identify or re-identify individuals. The work does not disclose an implementation bug, CVE, or vulnerability in a specific deployed product or service. It identifies an evaluation and design limitation in published LDPGNN protocols: their utility is often driven mainly by graph topology, and they frequently fail to outperform a feature-free baseline. Since no deployed system is tested or affected, coordinated vulnerability disclosure is not applicable.

References

- [1] W. L. Hamilton, R. Ying, and J. Leskovec, "Inductive Representation Learning on Large Graphs," in *NeurIPS*, 2017.
- [2] D. K. Duvenaud, D. Maclaurin, J. Aguilera-Iparraguirre, R. Gómez-Bombarelli, T. Hirzel, A. Aspuru-Guzik, and R. P. Adams, "Convolutional Networks on Graphs for Learning Molecular Fingerprints," in *NeurIPS*, vol. 28, 2015, pp. 2224–2232.
- [3] A. Fout, J. Byrd, B. Shariat, and A. Ben-Hur, "Protein Interface Prediction Using Graph Convolutional Networks," in *NeurIPS*, 2017, pp. 6530–6539.
- [4] L. Meng, Y. Bai, Y. Chen, Y. Hu, W. Xu, and H. Weng, "Devil in Disguise: Breaching Graph Neural Networks Privacy Through Infiltration," in *CCS*, 2023, pp. 1153–1167.
- [5] Z. Zhang, M. Chen, M. Backes, Y. Shen, and Y. Zhang, "Inference Attacks Against Graph Neural Networks," in *USENIX Security*, 2022, pp. 4543–4560.
- [6] X. Wang and W. H. Wang, "Group Property Inference Attacks Against Graph Neural Networks," in *CCS*, 2022, pp. 2871–2884.
- [7] Y. Shen, X. He, Y. Han, and Y. Zhang, "Model Stealing Attacks Against Inductive Graph Neural Networks," in *IEEE S&P*, 2022, pp. 1175–1192.
- [8] T. Wang, J. Blocki, N. Li, and S. Jha, "Locally Differentially Private Protocols for Frequency Estimation," in *26th USENIX security symposium (USENIX Security 17)*, 2017, pp. 729–745.
- [9] Ú. Erlingsson, V. Pihur, and A. Korolova, "RAPPOR: Randomized Aggregatable Privacy-Preserving Ordinal Response," in *CCS*, 2014, pp. 1054–1067.
- [10] B. Ding, J. Kulkarni, and S. Yekhanin, "Collecting Telemetry Data Privately," in *NeurIPS*, 2017.
- [11] N. Wang, X. Xiao, Y. Yang, J. Zhao, S. C. Hui, H. Shin, J. Shin, and G. Yu, "Collecting and Analyzing Multidimensional Data with Local Differential Privacy," in *ICDE*, 2019, pp. 638–649.
- [12] S. Sajadmanesh and D. Gatica-Perez, "Locally Private Graph Neural Networks," in *CCS*, 2021, pp. 2130–2145.
- [13] W. Lin, B. Li, and C. Wang, "Towards Private Learning on Decentralized Graphs with Local Differential Privacy," *IEEE Transactions on Information Forensics and Security*, vol. 17, pp. 2936–2946, 2022.
- [14] X. Pei, X. Deng, S. Tian, J. Liu, and K. Xue, "Privacy-Enhanced Graph Neural Network for Decentralized Local Graphs," *IEEE Transactions on Information Forensics and Security*, vol. 19, pp. 1614–1629, 2024.
- [15] H. Jin and X. Chen, "Gromov-Wasserstein Discrepancy with Local Differential Privacy for Distributed Structural Graphs," in *IJCAI*, 2022, pp. 2115–2121.
- [16] L. He, C. Li, P. Tang, and S. Su, "Going Deeper into Locally Differentially Private Graph Neural Networks," in *ICML*, 2025, pp. 22 548–22 565.
- [17] S. P. Kasiviswanathan, H. K. Lee, K. Nissim, S. Raskhodnikova, and A. Smith, "What Can We Learn Privately?" *SIAM Journal on Computing*, vol. 40, no. 3, pp. 793–826, 2011.
- [18] C. Dwork, F. McSherry, K. Nissim, and A. Smith, "Calibrating Noise to Sensitivity in Private Data Analysis," in *TCC*, 2006, pp. 265–284.
- [19] J. Gilmer, S. S. Schoenholz, P. F. Riley, O. Vinyals, and G. E. Dahl, "Neural Message Passing for Quantum Chemistry," in *ICML*, 2017, pp. 1263–1272.
- [20] T. N. Kipf and M. Welling, "Semi-Supervised Classification with Graph Convolutional Networks," in *ICLR*, 2017.
- [21] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph Attention Networks," in *ICLR*, 2018.
- [22] K. Xu, W. Hu, J. Leskovec, and S. Jegelka, "How Powerful Are Graph Neural Networks?" in *ICLR*, 2019.
- [23] S. Sajadmanesh and D. Gatica-Perez, "Locally Private Graph Neural Networks," in *CCS*, 2021, pp. 2130–2145.
- [24] Z. Li, R.-H. Li, M. Liao, F. Jin, and G. Wang, "Privacy-Preserving Graph Embedding Based on Local Differential Privacy," in *CIKM*, 2024, pp. 1–10.
- [25] Y. Bi, Z. Wang, Z. Cai, and X. Tong, "A Privacy-Preserving Mechanism Based on Local Differential Privacy in Edge Computing," *China Communications*, vol. 17, no. 9, pp. 50–65, 2020.
- [26] Z. Li, R.-H. Li, M. Liao, F. Jin, and G. Wang, "Privacy-Preserving Graph Embedding Based on Local Differential Privacy," in *CIKM*, 2024, pp. 1316–1325.
- [27] Z. Li, T. Wang, M. Lopuhaä-Zwakenberg, N. Li, and B. Škorić, "Estimating Numerical Distributions Under Local Differential Privacy," in *SIGMOD*, 2020, pp. 621–635.
- [28] Z. Yang, W. W. Cohen, and R. Salakhutdinov, "Revisiting Semi-Supervised Learning with Graph Embeddings," in *ICML*, vol. 48, 2016, pp. 40–48.
- [29] B. Rozemberczki and R. Sarkar, "Characteristic Functions on Graphs: Birds of a Feather, from Statistical Descriptors to Parametric Models," in *CIKM*, 2020, pp. 1325–1334.
- [30] B. Rozemberczki, C. Allen, and R. Sarkar, "Multi-Scale Attributed Node Embedding," *Journal of Complex Networks*, vol. 9, no. 2, p. cnab014, 2021.
- [31] N. Kriegeskorte, M. Mur, and P. Bandettini, "Representational Similarity Analysis – Connecting the Branches of Systems Neuroscience," *Frontiers in Systems Neuroscience*, vol. 2, p. 4, 2008.
- [32] S. Kornblith, M. Norouzi, H. Lee, and G. Hinton, "Similarity of Neural Network Representations Revisited," in *ICML*, vol. 97, 2019, pp. 3519–3529.
- [33] F. Ding, J.-S. Denain, and J. Steinhardt, "Grounding Representation Similarity Through Statistical Testing," in *NeurIPS*, 2021, pp. 1556–1568.
- [34] T. Cui, Y. Kumar, P. Marttinen, and S. Kaski, "Deconfounded Representation Similarity for Comparison of Neural Networks," in *NeurIPS*, vol. 35, 2022, pp. 19 138–19 151.
- [35] M. Raghu, J. Gilmer, J. Yosinski, and J. Sohl-Dickstein, "SVCCA: Singular Vector Canonical Correlation Analysis for Deep Learning Dynamics and Interpretability," in *NeurIPS*, 2017, pp. 6076–6085.

- [36] A. S. Morcos, M. Raghu, and S. Bengio, “Insights on Representational Similarity in Neural Networks with Canonical Correlation,” in *NeurIPS*, 2018, pp. 5727–5736.
- [37] X. Glorot and Y. Bengio, “Understanding the Difficulty of Training Deep Feedforward Neural Networks,” in *AISTATS*, 2010, pp. 249–256.
- [38] K. He, X. Zhang, S. Ren, and J. Sun, “Delving Deep into Rectifiers: Surpassing Human-Level Performance on ImageNet Classification,” in *ICCV*, 2015, pp. 1026–1034.
- [39] S. Ioffe and C. Szegedy, “Batch Normalization: Accelerating Deep Network Training by Reducing Internal Covariate Shift,” in *ICML*, 2015, pp. 448–456.
- [40] S. Santurkar, D. Tsipras, A. Ilyas, and A. Madry, “How Does Batch Normalization Help Optimization?” in *NeurIPS*, vol. 31, 2018.
- [41] D. Lei, Z. Song, Y. Yuan, C. Li, and L. Zhu, “Achieving Personalized Privacy-Preserving Graph Neural Network via Topology Awareness,” in *WWW*, 2025, pp. 3552–3560.
- [42] X. Zhu, V. Y. F. Tan, and X. Xiao, “Blink: Link Local Differential Privacy in Graph Neural Networks via Bayesian Estimation,” in *CCS*, 2023, pp. 2651–2664.
- [43] A. J. Haug, *Bayesian Estimation and Tracking: A Practical Guide*. John Wiley & Sons, 2012.
- [44] B. Perozzi, R. Al-Rfou, and S. Skiena, “DeepWalk: Online Learning of Social Representations,” in *KDD*, 2014, pp. 701–710.
- [45] A. Grover and J. Leskovec, “node2vec: Scalable Feature Learning for Networks,” in *KDD*, 2016, pp. 855–864.
- [46] C. Donnat, M. Zitnik, D. Hallac, and J. Leskovec, “GraphWave: Learning Structural Node Embeddings via Diffusion Wavelets,” in *KDD*, 2018, pp. 1321–1330.
- [47] Q. Lu and L. Getoor, “Link-Based Classification,” in *ICML*, 2003, pp. 496–503.
- [48] D. Lim, F. Hohne, X. Li, S. L. Huang, V. Gupta, O. Bhalerao, and S.-N. Lim, “Large Scale Learning on Non-Homophilous Graphs: New Benchmarks and Strong Simple Methods,” in *NeurIPS*, 2021, pp. 20 887–20 902.
- [49] H. Cui, Z. Lu, P. Li, and C. Yang, “On Positional and Structural Node Features for Graph Neural Networks on Non-Attributed Graphs,” in *CIKM*, 2022, pp. 3898–3902.
- [50] R. Abboud, İ. İ. Ceylan, M. Grohe, and T. Lukasiewicz, “The Surprising Power of Graph Neural Networks with Random Node Initialization,” in *IJCAI*, 2021, pp. 2112–2118.
- [51] R. Sato, M. Yamada, and H. Kashima, “Random Features Strengthen Graph Neural Networks,” in *SDM*, 2021, pp. 333–341.
- [52] M. Eliasof, F. Frasca, B. Bevilacqua, E. Treister, G. Chechik, and H. Maron, “Graph Positional Encoding via Random Feature Propagation,” in *ICML*, 2023, pp. 9202–9223.
- [53] G. Klambauer, T. Unterthiner, A. Mayr, and S. Hochreiter, “Self-Normalizing Neural Networks,” in *NeurIPS*, 2017.
- [54] D. P. Kingma and J. Ba, “Adam: A Method for Stochastic Optimization,” in *ICLR*, 2015.

Appendix A. Deferred Proofs

A.1. Proof of Theorem 1

Proof. For each node u , let $\Omega_u \subseteq \{1, \dots, d\}$ be the set of sampled feature dimensions. Under fixed- m MBM sampling, $|\Omega_u| = m$. For $j \in \Omega_u$, the LDP report satisfies $X_{uj}^* \in \{-1, 1\}$; for $j \notin \Omega_u$, we have $X_{uj}^* = 0$.

First, we record the row norm of the LDP report and the rectified report. Since each row of X^* has exactly m nonzero entries and each nonzero entry has absolute value 1,

$$\|x_u^*\|_2^2 = m.$$

The rectified report is

$$X' = \frac{d(e^{\varepsilon/m} + 1)}{m(e^{\varepsilon/m} - 1)} X^*.$$

Therefore, for every node u ,

$$\|x'_u\|_2^2 = \frac{d^2 (e^{\varepsilon/m} + 1)^2}{m(e^{\varepsilon/m} - 1)^2} = \alpha_{\varepsilon, m}.$$

Thus,

$$\text{Diag}(X'X'^\top) = \alpha_{\varepsilon, m} I_n.$$

Second, we decompose the inner-product matrix of X' into its diagonal and off-diagonal parts:

$$X'X'^\top = \text{Diag}(X'X'^\top) + H(X'X'^\top).$$

Using $\text{Diag}(X'X'^\top) = \alpha_{\varepsilon, m} I_n$, we obtain

$$X'X'^\top = \alpha_{\varepsilon, m} I_n + H(X'X'^\top).$$

Third, we apply the message-passing operator. Since $\hat{X} = QX'$, we obtain

$$\hat{X}\hat{X}^\top = QX'X'^\top Q^\top.$$

Substituting the previous decomposition gives

$$\begin{aligned} \hat{X}\hat{X}^\top &= Q(\alpha_{\varepsilon, m} I_n + H(X'X'^\top))Q^\top \\ &= \alpha_{\varepsilon, m} QQ^\top + QH(X'X'^\top)Q^\top. \end{aligned} \quad (15)$$

For the result of Equation 15, the first term is determined only by Q and the MBM parameters, and does not depend on the raw feature matrix X .

Fourth, we compute the conditional expectation of the off-diagonal term. For every coordinate j , MBM gives

$$\mathbb{E}[X_{uj}^* | X] = \frac{m}{d} C_{\varepsilon, m} X_{uj},$$

because dimension j is sampled with probability m/d , and the conditional mean of the sampled MBM sign is $C_{\varepsilon, m} X_{uj}$. Hence,

$$\mathbb{E}[X'_{uj} | X] = X_{uj}.$$

Equivalently, we obtain

$$\mathbb{E}[x'_u | X] = x_u.$$

For two different nodes $u \neq v$, their local randomizations are conditionally independent given X . Therefore,

$$\begin{aligned} \mathbb{E}[(X'X'^\top)_{uv} | X] &= \mathbb{E}[(x'_u)^\top x'_v | X] \\ &= \mathbb{E}[x'_u | X]^\top \mathbb{E}[x'_v | X] \\ &= x_u^\top x_v. \end{aligned}$$

Since $H(\cdot)$ removes diagonal entries, this gives

$$\mathbb{E}[H(X'X'^\top) \mid X] = XX^\top - \text{Diag}(XX^\top). \quad (16)$$

Multiplying by Q and $Q^\top$ yields

$$\mathbb{E}[QH(X'X'^\top)Q^\top \mid X] = Q(XX^\top - \text{Diag}(XX^\top))Q^\top.$$

Finally, define

$$\Delta_\varepsilon = H(X'X'^\top) - (XX^\top - \text{Diag}(XX^\top)).$$

Based on the expectation of $H(X'X'^\top)$ using [Equation 16](#), we have

$$\mathbb{E}[\Delta_\varepsilon \mid X] = 0.$$

Also,

$$H(X'X'^\top) = XX^\top - \text{Diag}(XX^\top) + \Delta_\varepsilon.$$

Substituting this identity into [Equation 15](#) gives

$$\hat{X}\hat{X}^\top = \alpha_{\varepsilon,m}QQ^\top + Q(XX^\top - \text{Diag}(XX^\top))Q^\top + Q\Delta_\varepsilon Q^\top.$$

This proves the theorem. $\square$

A.2. Proof of [Theorem 2](#)

Proof. We prove the result for the three mechanisms separately under a common argument. For mechanism $\mathcal{M}$, write

$$t_{MBM} = 2.18, \quad t_{PM} = 2.5, \quad t_{HDS} = 3.317.$$

The sampled dimension is

$$m_{\mathcal{M}} = \max\left(1, \min\left(d, \left\lfloor \frac{\varepsilon}{t_{\mathcal{M}}} \right\rfloor\right)\right).$$

We consider the regime $0 < \varepsilon \leq t_{\mathcal{M}}d$. In this regime, the cap $m_{\mathcal{M}} \leq d$ is not active except possibly at the boundary $\varepsilon = t_{\mathcal{M}}d$.

Let

$$s_{\mathcal{M}} = \frac{\varepsilon}{m_{\mathcal{M}}}.$$

We first show that $s_{\mathcal{M}}$ is bounded by a mechanism-dependent constant. If $0 < \varepsilon < 2t_{\mathcal{M}}$, then $m_{\mathcal{M}} \geq 1$, and therefore

$$s_{\mathcal{M}} = \frac{\varepsilon}{m_{\mathcal{M}}} \leq \varepsilon < 2t_{\mathcal{M}}.$$

If $\varepsilon \geq 2t_{\mathcal{M}}$, then $\varepsilon/t_{\mathcal{M}} \geq 2$. Since $\varepsilon \leq t_{\mathcal{M}}d$, we have $\lfloor \varepsilon/t_{\mathcal{M}} \rfloor \leq d$, and hence

$$m_{\mathcal{M}} = \left\lfloor \frac{\varepsilon}{t_{\mathcal{M}}} \right\rfloor.$$

For any $x \geq 2$, $\lfloor x \rfloor \geq x/2$. Thus,

$$m_{\mathcal{M}} \geq \frac{\varepsilon}{2t_{\mathcal{M}}},$$

which gives

$$s_{\mathcal{M}} = \frac{\varepsilon}{m_{\mathcal{M}}} \leq 2t_{\mathcal{M}}.$$

Therefore, in all cases,

$$0 < s_{\mathcal{M}} \leq 2t_{\mathcal{M}}.$$

We now rewrite each rectification multiplier as (d/ε) times a bounded one-dimensional function of $s_{\mathcal{M}}$.

For MBM, the rectification multiplier is

$$\beta_{MBM}(\varepsilon, d) = \frac{d(e^{\varepsilon/m} + 1)}{m(e^{\varepsilon/m} - 1)}.$$

With $s = \varepsilon/m$, this becomes

$$\begin{aligned} \beta_{MBM}(\varepsilon, d) &= \frac{d}{m} \cdot \frac{e^s + 1}{e^s - 1} \\ &= \frac{d}{\varepsilon} \cdot s \frac{e^s + 1}{e^s - 1}. \end{aligned}$$

Define

$$\phi_{MBM}(s) = s \frac{e^s + 1}{e^s - 1}.$$

This function is continuous on $(0, 2t_{MBM}]$ and has a finite limit at $s = 0$:

$$\lim_{s \rightarrow 0} s \frac{e^s + 1}{e^s - 1} = 2.$$

Therefore, $\phi_{MBM}(s)$ is bounded on $0 < s \leq 2t_{MBM}$. Then, we obtain

$$\beta_{MBM}(\varepsilon, d) = O\left(\frac{d}{\varepsilon}\right).$$

For PM, the rectification multiplier is

$$\beta_{PM}(\varepsilon, d) = \frac{d(1 + e^{-\varepsilon/(2m)})}{m(1 - e^{-\varepsilon/(2m)})}.$$

Again let $s = \varepsilon/m$. Then,

$$\begin{aligned} \beta_{PM}(\varepsilon, d) &= \frac{d}{m} \cdot \frac{1 + e^{-s/2}}{1 - e^{-s/2}} \\ &= \frac{d}{\varepsilon} \cdot s \frac{1 + e^{-s/2}}{1 - e^{-s/2}}. \end{aligned}$$

Define

$$\phi_{PM}(s) = s \frac{1 + e^{-s/2}}{1 - e^{-s/2}}.$$

This function is continuous on $(0, 2t_{PM}]$ and has a finite limit at $s = 0$:

$$\lim_{s \rightarrow 0} s \frac{1 + e^{-s/2}}{1 - e^{-s/2}} = 4.$$

Therefore, $\phi_{PM}(s)$ is bounded on $0 < s \leq 2t_{PM}$, and

$$\beta_{PM}(\varepsilon, d) = O\left(\frac{d}{\varepsilon}\right).$$

For HDS, the rectification multiplier is

$$\beta_{HDS}(\varepsilon, d) = \frac{d}{m \left(1 - \frac{1 - e^{-\varepsilon/m}}{\varepsilon/m}\right)}.$$

Let $s = \varepsilon/m$. Then,

$$\begin{aligned}\beta_{HDS}(\varepsilon, d) &= \frac{d}{m} \cdot \frac{1}{1 - \frac{1-e^{-s}}{s}} \\ &= \frac{d}{\varepsilon} \cdot \frac{s}{1 - \frac{1-e^{-s}}{s}}.\end{aligned}$$

Define

$$\phi_{HDS}(s) = \frac{s}{1 - \frac{1-e^{-s}}{s}}.$$

For every $s > 0$, $1 - e^{-s} < s$, so the denominator is positive. Moreover, $\phi_{HDS}(s)$ is continuous on $(0, 2t_{HDS}]$ and has a finite limit at $s = 0$:

$$\lim_{s \rightarrow 0} \frac{s}{1 - \frac{1-e^{-s}}{s}} = 2.$$

Therefore, $\phi_{HDS}(s)$ is bounded on $0 < s \leq 2t_{HDS}$, and

$$\beta_{HDS}(\varepsilon, d) = O\left(\frac{d}{\varepsilon}\right).$$

Combining the three cases proves the theorem. The constants hidden in the $O(\cdot)$ notation depend only on t_{MBM} , t_{PM} , and t_{HDS} , and do not depend on d or ε . $\square$

Appendix B. Additional Details of Experiments

We adopt the common GNN training settings: All GNN models have two graph convolutional layers, each with a hidden dimension of size 16 and a SeLU [53] activation function followed by dropout. The GAT model includes four attention heads.

Following existing works [12], [16], [26], we perform hyperparameter selection. The searched hyperparameter ranges are as follows: learning rate is chosen from $\{0.0001, 0.001, 0.01\}$, weight decay is chosen from $\{0.0, 0.0001, 0.001, 0.01\}$, dropout is chosen from $\{0.25, 0.5, 0.75\}$, the propagation step parameter K of both HOA and Kprop is chosen from $\{0, 2, 4, 8, 16, 32, 64\}$, the parameters τ of NFR layer in N-H architecture of UPGNET is chosen from $\{0.1, 0.3, 0.5, 0.7, 0.9\}$, the random feature dimension d_r of feature-free baseline is chosen from $\{800, 1600, 3200, 6400, 9600\}$, a range determined by the order of magnitude of feature dimensions in common datasets, which are shown in Table 3. We use the Adam optimizer [54] for all models. Without specification, we report the average results based on the best of each parameter selected by validation only. We measured the accuracy on the test set over 10 consecutive runs and report the average and 95% confidence interval calculated by bootstrapping with 1000 samples.